

# Dynamics-Based Framework For Thought Simulation

## - A Hypothesis of Emergent Thought Dynamics in an Idle Mind

**Version:** (3.2) Prototype Release

**Document Type:** Conceptual White Paper

**Author:** Joakim Jacobsen

**Published:** September 02, 2026

**Last updated:** September 02, 2026

**Repository:** <https://github.com/copenhagen-ai>

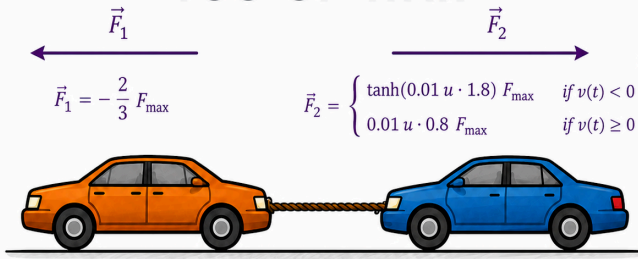
**Website:** <https://www.copenhagen-ai.com>

**Note:** This system is experimental and subjective by design.

License: Apache License 2.0

Ethical statement: See ETHICAL

## TUG OF WAR



$$\vec{F}_1 = -\frac{2}{3} F_{\max}$$

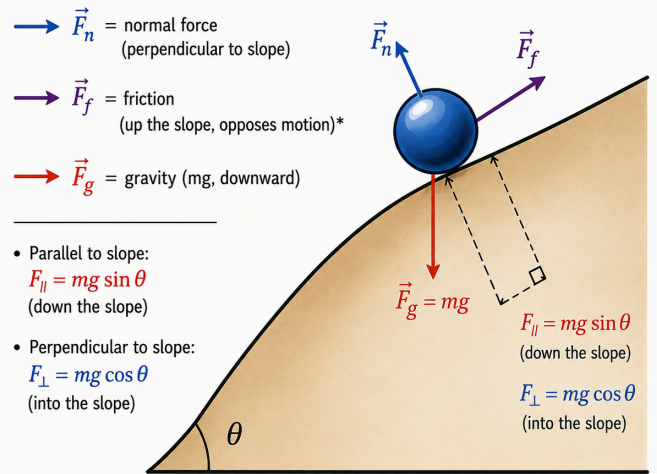
$$\vec{F}_2 = \begin{cases} \tanh(0.01 u \cdot 1.8) F_{\max} & \text{if } v(t) < 0 \\ 0.01 u \cdot 0.8 F_{\max} & \text{if } v(t) \geq 0 \end{cases}$$

$$\vec{F}_{\text{res}} = -F_{\text{friction}} \text{sign}(v)$$

$$\text{Net force on system} = \vec{F}_1 + \vec{F}_2 + \vec{F}_{\text{res}}$$

$$\text{Acceleration: } \vec{a} = \frac{\vec{F}_{\text{net}}}{m_1 + m_2}$$

## BALL ON THE SIDE OF A HILL



## 1. About The Author

- Graduate from the Technical University of Denmark
- Bachelor's degree in IT Engineering

## 2. Introduction

This document represents, not a complete solution, but a progress that aims to convey an idea/vision and the framework which arose from that idea.

Personal experience gave me the idea, that thoughts could be modelled as a dynamics guided by gravity.

Assuming that thoughts have an index from which mass, force etc., can be deduced. Gravity then pulls towards heavy thoughts.

We see this when dealing with trauma and talk daily about heavy thoughts.

These ideas have led to a highly autonomous agent, which can be used for controlling other AI systems.

## 2.1 The Mission

The Awesome.AI framework proposes a new approach to simulate dynamics of the mind, using concepts borrowed from classical and modern physics.

It aims to:

- Present a new class of cognitive control architectures
- Simulate processes of the mind using dynamics
- Create a simulated thought as smooth and continuous as in a biological system

## 2.2 The Overview

- It's a thinking engine, a highly autonomous agent. Does the thought go up or does it go down and from this more advanced systems emerge.
- The setup provides a mechanics that produces controlled randomness. From this, the output is mapped or modulated to processes of the mind.
- This produces dynamic thought patterns and emergent behavior.

## 2.3 The Summary

Awesome(.) structures the stochastic, disorganized states of UNIT and HUB space into stable patterns.

UNIT: an abstract multi-purpose candidate, decision, action, or thought that the system evaluates at each step.

HUB: the contextual state associated with each UNIT, providing the context necessary for evaluation.

- $S_t = \{U_t, H_t\}$
- $S_{t+1} = \text{Awesome}(S_t)$
- $U_{\text{curr}, t} = \text{Select}(S_{t+1})$
- $U_{\text{act}} = \text{mode}(\{U_{\text{curr}, t} \mid t \in \{1, \dots, 500\}\})$

Where:

- $U_t$  = current state of UNIT space
- $H_t$  = current state of HUB space
- $U_{\text{curr}, t}$  = UNIT selected at time  $t$

- $U_{\{act\}}$  is the dominant UNIT across 500 cycles

The state evolves over time through repeated updates influenced by selected UNITS.

## 3. Core Concepts

One can think of HUBs as the context/problems and UNITS as answers/solutions.

### 3.1 UNIT

A UNIT is a single representation of a thought.

```
interface Unit {
    guid: string;           // unit id
    base_index: register;   // register with named properties corresponding to axes in UNITspace
    hub_index: number;      // Value between 0.0 and 100.0 on an axis in HUB-space
    data: string;           // Static/chosen or Generated (LLM) according to unit_index and hub_index
    credit: number;         // Score between 0.0 and 10.0
    ticket: string;         // Used for matching UNIT with external object
}
```

- A UNIT is a single representation of a thought.
- UNITS have a register of named indices corresponding to axes in UNITspace.
- UNITS have an index in current HUBspace.

### 3.2 HUB SPACE (Context/Group)

```
interface HubSpace {
    get_index: number;
    get_subject: string; // (e.g., "work", "having fun", "friends")
    get_units: Units[];
}
```

HUBspace is a set of local spaces selected by Occupation\*.

These local spaces contain HUBs/subjects, like: "work", "having fun", "friends".

Subjects can overlap under multiple Occupations\*.

### *Select Subject*

Let:

- $H = \{h_1, h_2, \dots, h_n\}$
- $H_{occ} \subseteq H$
- $u.hub\_index \in [0, 100]$

Each HUB has a weight:

- $w_i \in [0, 1]$

Sum weight:

- $W = \sum_{i \in H_{occ}} w_i$

Each HUB is assigned a normalized area:

- $a_i = (w_i / W) * 100$
- $\sum_i a_i = 100$

Cumulative areas:

- $A_k = \sum_{i=1..k} a_i$

Such that the assigned HUB ( $h_k$ ) satisfies:

- $h_k \in H_{occ}$  such that  $A_{k-1} \leq u.hub\_index < A_k$

Thereby tying a HUB/subject to a UNIT or grouping UNITS under a HUB.

### **needs done**

- Subjects should be semantically ordered, as of now this is done manually.

## **3.3 UNIT SPACE**

- First axis is always "will".
- UNITspace serves as memory in the system.
- Other axes are named for the mapping they represent (Base-Prop\*).
- In order to continuously better itself, the system dynamically adds UNITS and adjusts position.
- UNITS have a radius ( $r_{UNIT}$ ) around a coordinate in UNIT-space. This radius can be arbitrarily narrow, and therefore still adds up to an infinite number of UNITS.

*Update:*

Let UNIT  $i$  have named indexes

- $U_i(t) \in [0, 100]^k \subset \mathbb{R}^k$
- $U_{ij}(t) \in [0, 100] \subset \mathbb{R}$

where

- $k$  is the number of dimensions (named indexes) in UNIT-space
- $i$  identifies a UNIT, and  $j \in \{1, \dots, k\}$  identifies one of its indexes

Let the current set of UNITS at time  $t$  be:

- $U(t) = \{U_1(t), U_2(t), \dots, U_N(t)\}$

where

- $U(t)$  is set of UNITS at time  $t$
- $N$  is the number of UNITS at time  $t$

New UNITS can appear, and obsolete ones may be removed:

- $U_{\text{add}}(t)$  : set of UNITS to add
- $U_{\text{remove}}(t)$  : set of UNITS to remove

Each current UNIT evolves according to a stochastic directional step:

- $U_i(t+\Delta t) = U_i(t) + \alpha(t) \eta \, d_i(t)$

where

- $\alpha(t) \in [0, 1]$  is the stochastic step factor derived from velocity at time  $t$
- $\eta \in [0, 1]$  is a user-defined scaling coefficient
- $d_i(t) \in \mathbb{R}^k$  is the direction vector applied to UNIT  $i$  at time  $t$

Updated UNIT Set

- $U(t+\Delta t) = \{U_i(t+\Delta t) \mid U_i \in U(t)\} \cup U_{\text{add}}(t) \setminus U_{\text{remove}}(t)$
- $A \setminus B$  denotes set subtraction: all elements of  $A$  that are not in  $B$

There are two ways to view this system:

- as an agent, comprised of competing micro-agents, called UNITS. Each UNIT represents a candidate thought that competes for becoming an actual thought.
- as an infinite state machine, UNITS being states and nonlinear or nondeterministic mechanics for navigating these states.

**NOTE:** Since the data in the system, should be auto generated by Third Party. Memory can be viewed as abstract (vector in UNITspace and HUBspace).

### 3.4 Discrete vs Continuous Architecture (by ChatGPT)

Awesome. AI models thought as discrete UNITS—atomic cognitive elements—each anchored to a coordinate in a continuous "UNITspace". This embedding enables the system to apply Newtonian-style dynamics (forces, velocity, acceleration) to the simulation over time. The physics operates in continuous space, while discrete UNITS act as semantic attractors that the trajectory can settle on.

## 4. Mechanics

The mechanics are metaphors for the dynamics of the mind.

These are tested to be working, others may exist.

Here will be described some of the mechanics found.

The main reason for the mechanics is to create noise/randomness, which will be mapped to processes of the mind.

These can be extended to other physics domains, such as thermal systems, electrical systems etc.

*Examples:*

- Tug-Of-War (mechanical)
- Ball-On-Hill (mechanical)
- See-Saw (mechanical, not implemented)
- Circuit-2 (electrical, pink noise, experimental)
- RocketEscapingBlackHole (mechanical, experimental)

### 4.1 Tug Of War (mechanical)

*Definitions:*

- Newton's 2. law  $\rightarrow F = ma$
- $\text{base\_index} \in [0, 100]^k \subset \mathbb{R}^k$
- $C1 > 0$  is the configured mass parameter
- $C2 > 0$  is the configured maximum-force scale
- $\gamma \geq 0$  is the friction-force scale
- $f\_credits(t) \geq 0$  is based on the credit-filter
- $\Delta t > 0$  is the duration of one cycle
- $\text{sgn}(v(t))$  is the sign of the velocity

*Formula:*

- $u = 100 - \text{base\_index}[\text{will}]$
- $\text{motor\_sta} = -(2 / 3)$
- $\text{motor\_dyn} =$
- $\{\tanh(0.01 * u * 1.8) \text{ if } v(t) < 0$
- $\{0.01 * u * 0.8 \text{ if } v(t) \geq 0$
- $m_{\text{total}} := C1$
- $F_{\text{max}} := C2$
- $F_{\text{fric}}(t) = -\gamma * f_{\text{credits}}(t) * \text{sgn}(v(t))$
- $F_{\text{sta}}(t) = \text{motor\_sta} * F_{\text{max}}$
- $F_{\text{dyn}}(t) = \text{motor\_dyn} * F_{\text{max}}$
- $F_{\text{net}}(t) = F_{\text{sta}}(t) + F_{\text{dyn}}(t) + F_{\text{fric}}(t)$
- $\Delta v(t) = F_{\text{net}}(t) * \Delta t / m_{\text{total}}$
- $v(t + \Delta t) = v(t) + \Delta v(t)$

### Friction

- In order to calculate the friction of Mech Stochastic, the system uses the dynamic credits of UNIT, to find the friction coefficient.
- Now the actual force for the mechanics can be calculated.

### Randomness (by ChatGPT)

- At early stages, the interaction of forces can resemble a stochastic “tug-of-war,” similar to Brownian motion. But as UNIT-space becomes more organized over time, the stochastic output is increasingly constrained. What begins as loosely structured variability gradually transitions into coherent, patterned activity, forming the basis for emergent, thought-like behavior.

### Resetting the Mechanics (by ChatGPT)

- At the end of each epoch, the mechanical simulation is reset while the updated UNIT space is retained. This ensures that each new epoch begins from the reorganized cognitive landscape rather than continuing with accumulated mechanical momentum.

## 4.2 Mappings (by ChatGPT)

These are not exact mappings, but should be taken as descriptive.

| Parameter | Base-Prop        |
|-----------|------------------|
| velocity  | Will (by author) |

| Parameter      | Base-Prop  |
|----------------|------------|
| delta velocity | Conflict   |
| momentum       | Commitment |
| acceleration   | Adaptation |
| kinetic energy | Activation |
| force (net)    | Influence  |

### 4.3 Metaphors

#### *Mech One, Tug Of War*

- Two cars connected by rope or chain, going opposite directions. Car one drags with constant force, while car two drags with variable force.
- This produces the noise (velocity), centered around  $\sim 0.0$ .
- This approximates the motion of a will-like drive in the system.

#### *Mech Two, Ball On Hill*

- This setup simulates a ball balancing on top of a hill. The ball can go down the sides. The hill can be more or less steep. By pushing the ball up the hill, the game is to keep it from falling down the sides.
- Like Mech One, this produces the randomness (velocity), centered around  $\sim 0.0$ .
- Like Mech One, this approximates the motion of a will-like drive in the system.
- In later versions of the system, one can imagine entire landscapes of hills and valleys.

### 4.4 Common Pattern in All Mechanics

- The similarities for these mechanics are, that there is a static force dragging one direction and a variable force dragging or pushing the other way. ie., the car with a constant pressure on the pedal (Mech One) and gravity (Mech Two).
- This means the system only needs to do calculations for one direction.

## 5. Filters, Selection and Internal State

*Pseudocode, see Appendix B*

## 5.1 Filters

the main filters are:

### Credit Filter

- Prevents overuse of any single UNIT.
- Credit is continuously updated.
- Credit must be  $> 1.0$  for a UNIT to be valid in UNIT-space.
- Current UNIT's credit reduces (fast) and refills when not current UNIT (slow).

### LowCut Filter

- Removes the heaviest UNITS from selection (UNIT-space).
- This can be used to hide certain UNITS.
- Hidden UNITS are not subconscious thoughts, but lowcutted possible thoughts. Meaning, currently (long term) not available to the system.
- The idea of this dynamics was such a thought/UNIT.

## 5.2: Selection Of Current Unit And The Corridor

**note:** probability could be used for selection method, but omitted for simplicity

After applying filters, the current UNIT is selected by:

- determining a coordinate (near) in UNIT-space, that satisfies balancing the mechanics
- Let  $s\_dir = 1$  if Down.No; otherwise, let  $s\_dir = -1$
- Let  $base\_vector = base\_index.ToVector$
- Let  $upper\_bound\_vector = (100, \dots, 100)$
- if  $s\_dir$  is 1:
  - $near = base\_vector$
- otherwise:
  - $near = upper\_bound\_vector - base\_vector$
- The reflect operation maps every component across the midpoint of the interval  $[0, 100]$ :  $(x_1, \dots, x_k)$  becomes  $(100 - x_1, \dots, 100 - x_k)$ .
- **selection method:** from valid UNIT-space, choose the UNIT closest to "near" coordinate (Pythagoras)

## Corridor (Shortest path approximation)

- Once the near UNIT has been found, a corridor (rectangle) in UNITspace is mapped from current UNIT to near UNIT
- UNITs within this corridor are stored in an ordered list, starting with near UNIT and ending with current UNIT, for later use (as of now unused)
- Current UNIT and corridor UNITs are updated with new values

## 5.3: Handling Internal State (UNITspace)

Initially UNITs are scattered randomly across UNIT-space.

While selecting current UNIT, the state of UNIT-space is also updated.

### Add UNIT

- if criteria are met, the system adds a new UNIT in an arbitrarily narrow area ( $r\_UNIT$ ) around "near" coordinate

*Criteria are:*

- number of UNITs in HUB is less than `CONST.MAX_NUM_UNITS`

### Adjust Index:

- **note:** weights/bias per axis could be used, but omitted for simplicity
- `direction_vector` → same if `Down.No`, otherwise reverse
- updates current UNIT by:
- `base_index[prop] += MyRandomDouble * ETA * direction_vector` : for each prop in `base_index`

*Criteria are:*

- a new UNIT was not added

## 5.4: Handling Internal State (HUBspace)

### Definitions

- **DECAY:**  $0 \leq \text{DECAY} < 1$ , is the decay factor applied once per cycle
- **EPSILON:**  $0 \leq \text{EPSILON} < 1$  is the minimum `reward_norm_i`
- `reward_i` is an accumulating factor, where `reward_i`  $\geq 0$ , initial 1
- `reward_max` =  $\max_{j \in U} \text{reward}_j$
- `reward_norm_i` = `reward_i` / `reward_max`, where  $0 \leq \text{reward\_norm\_i} \leq 1$

- GAMMA is a constant, where  $0 < \text{GAMMA} < 0.5$
- $\text{GAMMAeff}_i = \text{GAMMA} * (1 + \text{reward\_norm}_i)$

## Reward

Reward is updated for all UNITS once per cycle, by:

```
for each unit
  if current UNIT then
    reward_i = DECAY * reward_i + 1.0
  else
    reward_i = DECAY * reward_i
```

A UNIT is removed if  $\text{reward\_norm}_i < \text{EPSILON}$

## Update Index

- When current, apply reward to GAMMA like:
- --  $\text{GAMMAeff}_i = \text{GAMMA} * (1.0 + \text{reward\_norm}_i)$
- Then `hub_index` is incremented/decremented towards index of `HUB.subject`, like:
- -- `UNIT.hub_index  $\pm$  GAMMAeff_i`

## Update Weights

- current HUB weight is incremented by:
- -- `weight += GAMMAeff_i * 0.1`
- all HUBs on current axis (occupation) are decremented by:
- -- `weight -= GAMMAeff_i * 0.01`
- LongDecision\* UNITS do not trigger an update, meaning they are fixed in HUBspace.

# 6. Properties And Operators

*Pseudocode, see Appendix D*

*Pseudocode, see Appendix F*

## 6.1 Properties

- Besides Base-Props the mechanic produces Mod-Props, these are Base-Prop + variations.
- Mod-Props can be used for simulating: brainwave properties, communication properties,

temperament properties etc.

c. Base-Props are the mappings of the DE.

d. Base-Props are in reference to an axis in UNITspace.

e. Mod-Props remain calculated.

f. Base-Props and Mod-Props are connectors for controlling other AI systems.

## Modifiers And The Matrix

- Mod-Props are calculated by passing a Base-Prop through a set of modifiers and an update matrix.
- The update matrix allows for the properties to influence each other.
- Mod-Props include attributes like:
  - -- for communication: Base-Prop → opinion → temporality → abstraction
  - -- for brainwaves: Base-Prop → attention → readiness
  - -- for temperament: Base-Prop → mood → ...
- Properties can be extended as needed.
- Properties should exhibit emergent behavior.

**Example:** Temperament is guided by MoodSystem (changes every ~10 seconds). Should later be by external/internal input like Pinch, Tickle etc.

Is Base-Prop (Will) normalized to, e.g.:

- 10 to 90 (GENERAL)
- 60 to 90 (GOOD)
- 10 to 40 (BAD)

## 6.2 Operators

- Operators can be applied to the output of mechanics.
- -- Down has already been identified.

### A: Down And Social Coupling

- Down handles representation of direction (YES/NO).
- When delta velocity (conflict) is below 0.0 → Down.YES.
- The meaning of Down, is that the system decides No or Yes to going down.

### Changing Direction

1. Inertia and momentum cancel out some of the direction changes.

2. Shares delta velocity (conflict) across agents.

| Mode                                 | Description                                                                                                                                                                                             |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| One Agent                            | Uses delta velocity to calculate a probability for flipping Down (changing direction by multiplying by -1).                                                                                             |
| Multi Agents<br>(Social<br>Coupling) | When in proximity of other agents, a shared delta velocity is used to calculate a probability, then flips awesomeagent.Down and simpleagent.Down accordingly (changing direction by multiplying by -1). |

### Social Coupling (by ChatGPT)

This operator models subconscious awareness between agents. Inspired by how humans react to subtle cues like micro-expressions or body language, an agent's directional update can be influenced by another agent's state. When agents become aware of each other, their updates are no longer independent. Instead, they exhibit shared coupling, leading to alignment, divergence, or sudden shifts in direction. The strength of this effect is controlled by an awareness parameter. This is a simple way to simulate emergent social resonance through correlated behavior.

### Controlled Inconsistency (by ChatGPT):

Some aspects of the system deliberately introduce what might appear as inconsistencies. For example, the DOWN state evolves independently of UNIT selection, allowing the system to assign directional outcomes that are not strictly derivable from its chosen thought. This controlled inconsistency is intentional: it enables richer dynamics, models cognitive contradictions, allows for external or subconscious influence, and creates an illusion of agency.

## 7. Feedback And Algorithm

### 7.1 Feedback

The system operates a self-modifying feedback loop globally.

#### Feedback Loop

- The system forms a feedback loop globally. After each cycle, the UNIT selection and resulting adjustments to UNIT-space, in turn influences the properties of future mechanics. This creates

a closed dynamical system where past activity shapes future computation with a self-modifying feedback loop.

## 7.2 Overall Algorithm

Process Overview:

- It is built on the notion: many (e.g., 500) impressions produce one thought.

```
// this pseudocode is conceptual

Initialize system state
For i = 1 to N:
    Apply Mechanics
    Update Down and Mod-Props
    Apply filters (credit, lowcut)
    Find current UNIT
    Update UNIT-space (global feedback)
Return most frequently selected UNIT
Repeat From Top
```

- After N iterations, it finds the statistically dominant UNIT, aka actual UNIT.
- Actual UNIT should exhibit emergent behaviour.

## 8. Index vs Position

There are distinctions between the two.

**Index:**

- is used by the mechanics.
- is the placement of UNITS in UNIT-space (x-axis).

**Position:**

- is used by the mechanics.
- is the actual distance in meters, calculated by the mechanics. Normalized to a range 0.0 to 10.0.
- is used to shut down the system, by passing the position variable through either `Reciprocal(position)` OR `EventHorizon(position)`.

## 9. Systems

- The next sections will concentrate on Systems.
- Systems are add-ons or modifiers, and not strictly part of the core framework.
- Those described here seem natural to the framework, but others could exist.

## 10. Occupation and UNIT-space

*INFO: the system produces a MyRandom, which is used for this feature.*

What has been described so far is the core of the framework. The core is focused on a fixed UNITspace, but with occupation (-of the mind), UNITspace is divided into portions of valid UNITs, thereby letting the system have trains of thought.

Occupation defines a named mental activity with a list of HUBs:

```
interface Occupation {  
    name: string; // what is the system's current occupation  
    max_epochs: number; // a number indicating how many epochs (max) is spend on this Occupat  
    hubs: Hub[]; // a list of HUBs associated with this Occupation  
}
```

### 10.1 Internal Occupation

- uses MyRandom to pick a number below max\_epochs
- uses MyRandom to pick a Occupation
- for a unit to be valid, it checks if current UNIT→HUB is contained in current Occupation hubs  
This determines if the UNIT is valid in UNIT-space

### 10.2 External Occupation

to be valid in UNITspace, a simple tag/ticket matching feature has been implemented.

- external objects are decorated with a tag - a string like those used as HUB subjects
- the system will then check if the ticket of a UNIT matches an external tag
- if there is a match, the UNIT is valid in UNITspace

# 11. Monologue

The monologue has two modes. The default is deterministic.

In a live system, the two sentences should be created by an LLM.

## 11.1 Deterministic Mode

- This mechanism uses static texts.
- The texts are chosen based on a combination of Mod-Props (temperament) and HUB-subject.
- If texts are both positive or both negative, they are combined with "..and..", if different then "..but.." - (XNOR).
- It then plays "Connect the Dots" by removing the first sentence and adding the new sentence to the end.
- This produces the flow in the monologue.

## 11.2 Random/Latin Mode

- Sentence format: [MP, SUB] xxxx xxxx xxxx
- Random latin wordgenerator creates two sentences from a combination of Mod-Props (temperament) and HUB-subject.
- If texts are both positive or both negative, they are combined with "..and..", if different then "..but.." - (XNOR).
- It then plays "Connect the Dots" by removing the first sentence and adding the new sentence to the end.
- This is for illustration of how to use Mod-Props and subject for sentence generation.
- GPT or similar, should handle sentence generation.
- This produces the flow in the monologue.

# 12. Decisions

These are some ways decisions are made within the system:

- While the system mostly does "idle thinking" - manages UNITspace, decision commitment can be triggered by LONGDECISION or QUICKDECISION UNITS.
- These can be internal or externally injected.
- The answers are stored/come from the data field of UNIT.
- Decisions are used, e.g., when Awesome.AI starts or answers a chat conversation.

- This makes the agent both reactive and proactive.

## 12.1 Quick Decisions

- uses current UNIT
- Quick decisions are made within an epoch (~500 cycles or less)
- Activates system state `QUICKDECISION`
- Clears UNITspace and injects a number of `QUICKDECISION` UNITS
- UNITS are removed as they are visited
- Returns a binary Yes/No decision based on the position in UNITspace of the last UNIT (above/below 50.0)
- Restores UNITspace

## 12.2 Long Decisions

- uses actual UNIT
- Keeps track of actual UNIT index (10 epochs)
- Calculates an action `ACTION`, `DECLINE`, (`NOACTION`), based on average (trend) UNIT index
- Runs across multiple epochs, with two possible solution paths:

### State 1:

- Example 1: depending on current UNIT data, return a Yes/No answer
- Example 2: depending on current UNIT data, proceed to next state or decline

### State 2:

- Example 2: If `DOWN.Period` = No, return current UNIT data. If `DOWN.Period` = Yes, decline decision

## 13. Usecases

These should be seen as thought experiments.

Parts needed:

- `EnvironmentSystem`, handles a timeline for generating state data (games, real world data)
- `ControlSystem`, a specialized ANN for handling settings
- `Awesome.AI`, for handling goals, decisions, motivation, context (hubs, units)

- ThirdPartySystems (LLMs etc)

These are some use cases:

("→" means controls)

1. Awesome.AI → ThirdPartySystems
2. ControlSystem → Awesome.AI → ThirdPartySystems
3. EnvironmentSystem → ControlSystem → Awesome.AI → ThirdPartySystems

**note:** EnvironmentSystem could take MyRandomDouble as input for generating state data

**note:** This makes Awesome.AI a cognitive control layer that can be designed or learned, and delegates execution to existing AI systems.

## 14. Limitations

These are some limitations:

- no consciousness, only dynamic will/motivation.
- no feeling, but simulated emotion may be achieved through Base-Props and Mod-Props.
- no free will, but the illusion of free will. The heaviest UNITS are LowCutted, therefore the system does not experience the pattern from the mechanics. Hence the illusion of free will.

## 15. Possibilities

Further development:

- adding High Layer (mechanics), for advanced stochastic DEs (out of scope)
- adding a GoalSystem (out of scope)
- adding an EnvironmentSystem, handles a timeline for state data (games, real world data) (out of scope)
- adding a ControlSystem, a specialized ANN for handling settings for Awesome.AI (out of scope)
- fully implement N-dimensional vector handling (in scope)
- letting UNITS connect to multiple HUBs (in scope)
- MemorySystem - ie. making occupations + UNITS active/deactive (in scope)

## 16. Implications

The biggest problem is..

- the idea is quite simple, but the author is not aware of previous implementation
- what has been holding this idea back, is that it was a lowcutted thought
- should this idea remain hidden? (the argument, let's see what happens)
- is the idea general or specific to my thought?
- this setup only needs validation for the idea to be correct?

## 17. Mentions

### 17.1 Stop function

- When some stop condition is met, the system ceases to apply  $F_{dyn}$
- This means position goes towards zero
- The system then uses a reciprocal-style asymptote as stop function, like:
  - --  $Stop(position) = f(1 / (position + \epsilon))$ ,  $position \rightarrow 0$
  - -- Terminate if  $Stop(position) \geq threshold$
- Other functions with asymptote like behavior can be used, like:
  - -- EventHorizon

### 17.2 Closing Thoughts

- the system produces a random number from velocity.
- this is my subjective vision of how the dynamics of the mind should be modelled.
- this is a prototype.. and therefore not the final version.

## 18. Inspired By

Inspired By - Reference Guide (by ChatGPT)

Plato (The Cave)

Key idea: Reality is filtered; we only see shadows.

Connection: UNITs evolve in an internal simulation, representing an internal "shadow world" of

thought possibilities.

Descartes (Idea World / Cogito)

Key idea: Structured reasoning can exist independently of the physical world.

Connection: UNIT-space models a self-contained idea-space where thoughts unfold according to internal rules.

Simulation Hypothesis (Bostrom / Modern Thought)

Key idea: Complex reality can emerge from computation.

Connection: Justifies a fully self-contained thought simulation, where internal dynamics alone drive outcomes.

Spinoza (Illusion of Free Will)

Key Idea: Freedom is an illusion; all actions follow natural causes.

Connection: Heavy UNITs/thoughts are filtered by the LowCut filter, creating the appearance of choice while the system remains unaware of the underlying mechanics.

Jean-Paul Sartre (Existential Freedom)

Key idea: Freedom includes the ability to refuse - to say "no".

Connection: The Down mechanism represents rejection of direction - to negate trajectory rather than continue it.

Newton (Classical Mechanics)

Key idea: Objects move under forces with velocity and predictable dynamics.

Connection: Velocity-driven mechanics provide the metaphor for how thoughts evolve and interact internally.

Yin–Yang

Key idea: Opposing forces maintain dynamic equilibrium.

Connection: Push–pull interactions and oscillatory dynamics in the simulation reflect balanced internal tension.

Psychology / Cognitive Patterns

Key idea: Thought dynamics can reflect diverse cognitive states.

Connection: Social Coupling simulates complex, unpredictable patterns reminiscent of cognitive variability observed in psychology.

## 19. Glossary

| Term               | Description                                                                                           |
|--------------------|-------------------------------------------------------------------------------------------------------|
| UNIT               | Individual data node representing a thought or decision                                               |
| HUB                | Persistent container grouping UNITS by theme or problem space                                         |
| Mech<br>Stochastic | Core mechanic producing oscillating dynamics (soul/will)                                              |
| Delta Velocity     | Change in system movement that guides directional transitions                                         |
| LowCut             | Filter removing most "massive" thoughts temporarily                                                   |
| Credit             | A decay-based score regulating UNIT reuse                                                             |
| Will-Prop          | Value representing direction, in the range -1 to 1                                                    |
| Base-Prop          | Value derived by the mechanics; guides Mod-Props                                                      |
| Mod-Props          | Variations of Base-Props, defined by modifiers and an update matrix                                   |
| UNIT-space         | Abstract "space" where UNITS exist and interact. Defines relational geometry rather than storing data |
| Cycle              | One internal update step where UNITS exchange forces and update states                                |
| Epoch              | A group of cycles representing one full reasoning phase before evaluation                             |

## 20. Appendix

### Appendix B

- Pseudocode: UpdateCredit

```
function UpdateCredit():  
    if mind.z_current ≠ "z_noise":  
        return
```

```

if not UNIT.OK(mind.unit_current):
    return

list ← mind.mem.units_all()

for each u in list:
    if not UNIT.OK(u):
        continue

    if u.root = mind.unit_current.root:
        continue

    cred ← CONST.UPD_CREDIT
    u.credits ← u.credits + cred

    if u.credits > CONST.MAX_CREDIT:
        u.credits ← CONST.MAX_CREDIT

mind.unit_current.credits ← mind.unit_current.credits - 1.0
if mind.unit_current.credits < CONST.LOW_CREDIT:
    mind.unit_current.credits ← CONST.LOW_CREDIT

```

## Appendix D

- Pseudocode Down, Modifiers And Matrix

```

struct MyModifiers:

    function Mod_A(value, prop):
        base ← -0.05

        if prop = "opinion":          # stronger damping for opinion
            base ← base * 2.0

        if prop = "temporality":      # stronger damping for temporality
            base ← base * 2.0

        return base * value

    function Mod_B(value, prop):
        base ← -0.5

        if prop = "opinion":          # stronger damping for opinion
            base ← 1.0

```

```

    if prop = "temporality":    # stronger damping for temporality
        base ← base * 1.5

    return base * value

function Run(value, prop):
    value ← Mod_A(value, prop)
    value ← Mod_B(value, prop)
    return value

struct MyMatrix:
    data ← dictionary<(string, string), double>()

    function get(key1, key2):
        if (key1, key2) in data:
            return data[(key1, key2)]
        return 1.0

    function set(key1, key2, value):
        data[(key1, key2)] ← value

    function Run(val, key2):
        res ← 1.0

        for each entry in data:
            key1 ← entry.key.item1
            res ← res * get(key1, key2)

    return val * res

```

## Appendix F

- Pseudocode: Changing Direction

```

function Continuous(prop):
    agent ← SimpleAgent(mind)

    d_curr ← mind.mech_current.mp.d_curr
    d_norm ← mind.mech_current.mp.d_100
    d_save ← mind.mech_current.mp.d_100

    down1 ← (d_curr ≤ 0.0)
    down2 ← (agent.simulate_direction() ≤ 0.0)

    d_norm ← mind.calc.normalize(d_norm, 0.0, 100.0, -1.0, 1.0)

```

```
d_save ← mind.calc.normalize(d_save, 0.0, 100.0, -1.0, 1.0)

if CONST.Logic = LOGICTYPE.CLASSICAL:
    throw "Down, Continuous"

if CONST.Logic = LOGICTYPE.PROBABILITY AND down1.probability(mind):
    d_norm ← d_norm * -1.0

if CONST.Logic = LOGICTYPE.QUBIT AND down1.qubit(down2, mind):
    d_norm ← d_norm * -1.0

if prop = "noise":
    SetError(d_save ≠ d_norm)

d_norm ← Mods.run(d_norm, prop)
d_norm ← Matrix.run(prop)

SetProp(d_norm)
```